

1st Lab with VTK (Visualization Tool Kit)

João Varela, 113780
Carolina Prata, 114246
Information Visualization, 2025
MSc in Computer Engineering, University of Aveiro

Abstract—This document provides an overview of the work carried out during the first laboratory session of the Information Visualization course, whose purpose was to gain an initial understanding of the VTK framework.

I. INTRODUCTION

The Visualization Toolkit (VTK) is an open-source software framework extensively used in 3D computer graphics, image processing, and scientific visualization. It offers a robust collection of tools for constructing, modifying, and rendering complex visual representations. VTK handles numerous data types and supports various visualization methods, including volume rendering, contouring, and streamline analysis.

II. DEVELOPMENT

The laboratory assignment provided by the course instructor comprises several exercises covering the following topics:

- VTK installation;
- First VTK example and the visualization pipeline;
- VTK interactors;
- Camera control;
- Lighting;
- Actor properties;
- Final exercise.

A. VTK installation

For this exercise, it was only required to install VTK in our local machine, and for that we used `pip` inside a virtual environment. The `requirements.txt` file includes the required `vtk` package. The commands executed were:

```
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

B. First VTK example and the visualization pipeline

In this section, several introductory exercises are provided. The first task consists of downloading and running the file `cone.py`, which contains Python code illustrating a basic visualization example using VTK. When executed, the script generates the following output:

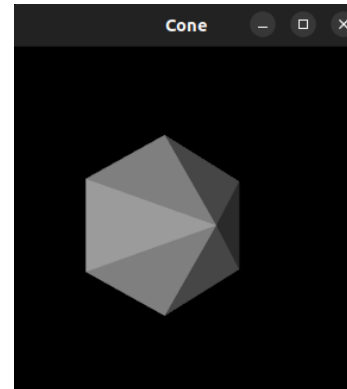


Fig. 1. Result of executing the `cone.py` example.

In this exercise, we were asked to modify the cone defined in `cone.py` by adjusting its geometric parameters. Specifically, we updated the code to set the cone's height to 2 and its radius to 1, using the following instructions:

```
coneSource.SetHeight(2)
coneSource.SetRadius(1)
```

Additionally, we were instructed to run several tests by applying different values to `SetResolution`, allowing us to observe how changes in resolution affect the visual representation of the cone. Here are some example outputs obtained during these tests.

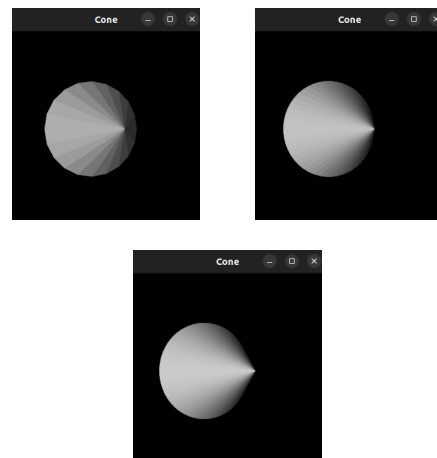


Fig. 2. Cones with resolutions of 20, 50 and 100.

The `SetResolution` method is used to control the number of facets used to approximate the cone's circular base, effectively

defining how smooth or coarse the cone appears in the rendered visualization.

It was also requested to change the background color of the rendering window to white. This was achieved by applying the `SetBackground(1,1,1)` method to the renderer.

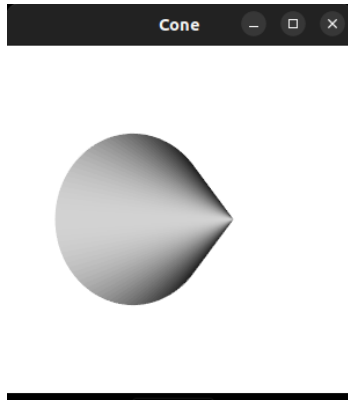


Fig. 3. Cone with the background changed to white

The final exercise consisted of modifying the size of the visualization window and determining its default dimensions. To achieve this, we added the instruction `renWin.SetSize(x, y)`, where `x` and `y` define the desired window width and height. We tested the sizes `300×300`, `400×400` and `500×500`, and observed that the default window size corresponds to `300×300`.

In this exercise, we were asked to extend the original code by adding two additional geometric primitives: a sphere and a cylinder. The sphere was implemented using `vtkSphereSource` with a radius of 2, while the cylinder was created through `vtkCylinderSource` with a radius of 2 and a height of 3.

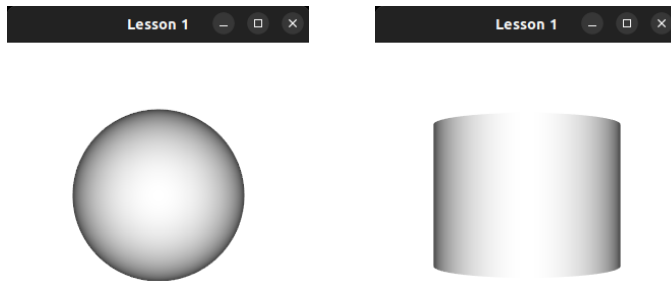


Fig. 4. Sphere and Cylinder

C. VTK Interactors

In this section, we were instructed to explore the interactor provided by the VTK package. For this purpose, we replicated the previous example into a new file named `interaction.py`, and added the following lines:

```
iren = vtkRenderWindowInteractor()
iren.SetRenderWindow(renWin)
```

```
iren.Initialize()
iren.Start()
```

All the interaction controls listed in the assignment README were tested, and we confirm that their behavior matches the expected effects.

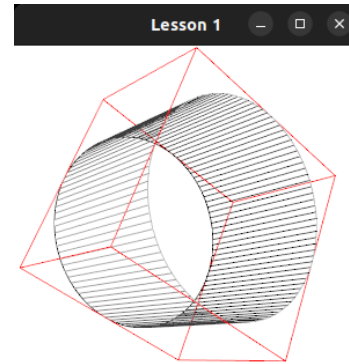


Fig. 5. Example of interactions (Wireframe and Pick).

D. Camera Control

In this section, we explored different aspects of camera control in VTK. For this purpose, we replicated the previous example into a new file named `camera.py`. Following the instructions, we added the provided camera configuration code immediately after creating the renderer:

```
cam1 = vtkCamera()
cam1.SetPosition(10,0,0)
cam1.SetViewUp(0,1,0)
ren.SetActiveCamera(cam1)
```

We then modified the camera parameters by setting the *Position* to `(10, 10, 0)` and the *View Up* vector to `(0, 1, 1)`. After applying these changes, the visualization adjusted accordingly, producing a slightly oblique view of the cylinder. By further experimenting with different camera positions and orientations, we verified how each parameter affects perspective, depth, and the overall perception of the rendered scene.

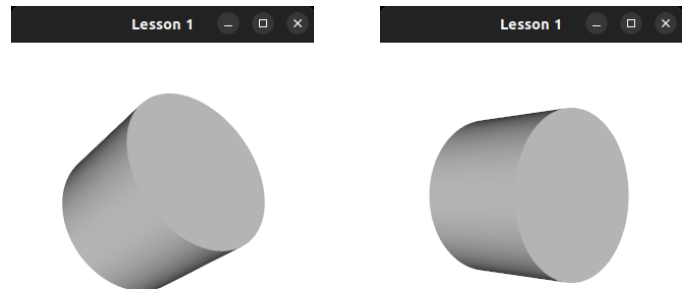


Fig. 6. Different views of the camera, the first one with the vector `(0,1,1)` and the second one with the `(0,0,2)`

In this part of the exercise, instead of creating a new camera using `cam1 = vtkCamera()`, we were in-

structed to retrieve the default camera created by the renderer. This was done by replacing that line with `cam1 = ren.GetActiveCamera()`, allowing us to modify the existing camera rather than creating a new instance.

Afterwards, we enabled the orthographic projection mode using `cam1.SetParallelProjection(True)` and visualized a cube in wireframe. Switching between perspective and orthographic projection revealed clear differences: in orthographic mode, the cube is rendered without perspective distortion, and its parallel edges remain parallel regardless of distance.

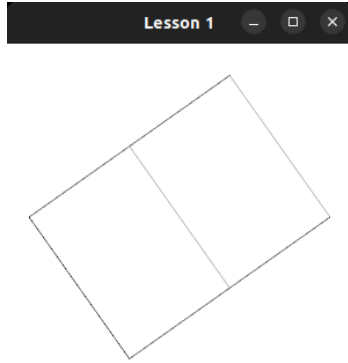


Fig. 7. Cube (wireframe) in paralel projection.

E. Lighting

This section focused on experimenting with lighting in VTK. For this purpose, we replicated the previous example into a new file named `lightning.py`. To achieve this, we introduced a new light source and configured it based on the camera's parameters. The following lines were added to create a red light, position it at the camera's location, and attach it to the renderer:

```
cam1 = ren.GetActiveCamera()
light = vtkLight()
light.SetColor(1,0,0)
light.SetFocalPoint(cam1.GetFocalPoint())
light.SetPosition(cam1.GetPosition())
ren.AddLight(light)
```

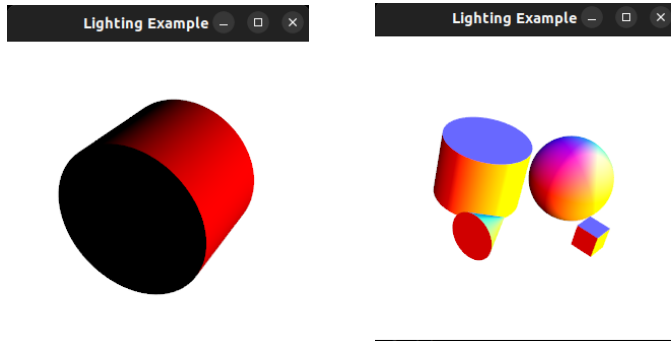


Fig. 8. Examples of lights in different shapes.

The second image shows the scene illuminated with multiple light sources of different colors. We applied four lights: red, green, yellow, and blue, using all the geometric figures created so far to observe the combined lighting effects.

F. Actor Properties

It was also requested to modify the visual properties of an actor. For this purpose, we used the original `cone.py` file and created a new file named `actors.py`. To do this, we adjusted both the color and the opacity of the cone using the following code instructions:

```
coneActor.GetProperty().SetColor(0.2, 0.63, 0.79)
coneActor.GetProperty().SetOpacity(1)
```

These changes allow the actor to be rendered with a customized color and transparency level, giving finer control over the object's appearance.

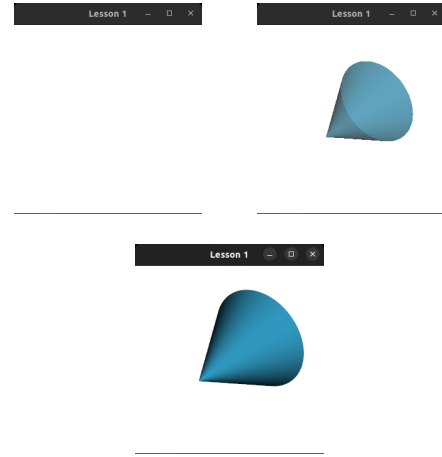


Fig. 9. Cone visualization with the applied color and opacity settings (0, 0.5, 1).

G. Last Exercise

In the final exercise, we were asked to construct a multi-light environment by adding four colored light sources, each positioned in a different location in the 3D space and all directed towards the origin. The required lights were:

- Red light at position $(-5, 0, 0)$,
- Green light at position $(0, 0, -5)$,
- Blue light at position $(5, 0, 0)$,
- Yellow light at position $(0, 0, 5)$.

To avoid code repetition, we implemented a function `create_light_with_sphere(renderer, color, position)` that receives a color and a position, creates a `vtkLight` oriented towards the origin using `SetFocalPoint(0,0,0)`, and places a small sphere at the corresponding coordinates to visually represent the light's location. These spheres were assigned a radius of 0.5 and lighting was disabled through `actor->GetProperty()->SetLighting(False)` so that they preserve their true color and are not affected by other light sources.

To observe the combined effect of the four lights, we placed two different primitives at the center of the scene: first a sphere (with radius 2), and later a cone (with height 2 and radius 1). The images included below illustrate the visual differences between these two central objects when illuminated by the same multi-light configuration.

The results show how each surface reacts to directional colored lighting coming from four distinct positions. The sphere exhibits smooth shading transitions between colors, while the cone produces stronger contrasts due to its polygonal structure. In both cases, the colored spheres representing the lights retained their pure red, green, blue, and yellow tones, confirming that disabling lighting for these actors worked as intended.

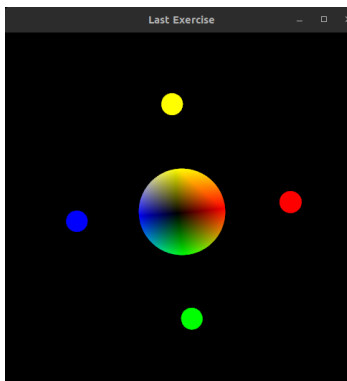


Fig. 10. Scene illuminated by four colored lights using a sphere as the central object.

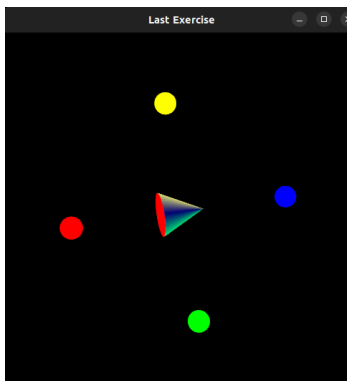


Fig. 11. Attempt to replicate the reference example `lighting.png` using a cone as the central object under the same multi-light configuration.